TP 3

Les variables d'environnement et les programmes Set-UID

1. Introduction

L'objectif de ce TP est de permettre aux étudiants de comprendre comment les variables d'environnement affectent le comportement des programmes et du système. Les variables d'environnement sont un ensemble de valeurs dynamiques nommées qui peuvent affecter la façon dont les processus en cours d'exécution se comportent sur un ordinateur. Elles sont utilisées par la plupart des systèmes d'exploitation, depuis qu'elles ont été introduites dans Unix en 1979. Bien que les variables d'environnement affectent le comportement des programmes, la manière dont elles y parviennent n'est pas bien comprise par de nombreux programmeurs. Par conséquent, si un programme utilise des variables d'environnement, mais que le programmeur ne sait pas qu'elles sont utilisées, le programme peut présenter des vulnérabilités.

Dans ce TP, nous essayons de comprendre comment ces variables fonctionnent, comment elles sont propagées du processus parent au processus fils, et comment elles affectent les comportements du système/programme ; notamment les programmes Set-UID, qui sont habituellement des programmes privilégiés.

Environnement de laboratoire : VM SEED Ubuntu 20.04.

2. Application

Les fichiers nécessaires pour ce laboratoire sont inclus dans le dossier `Labsetup.zip`.

2.1. Manipulation des variables d'environnement

Dans cette activité, nous étudions les commandes qui peuvent être utilisées pour définir et supprimer les variables d'environnement. Nous utilisons Bash dans le compte seed. Le shell par défaut qu'un utilisateur utilise est défini dans le fichier /etc/passwd (le dernier champ de chaque entrée).

- Commande pour voir le shell par défaut : cat /etc/passwd | grep seed
- Utilisez la commande *printenv* ou *env* pour imprimer les variables d'environnement. Si vous êtes intéressé par certaines variables d'environnement particulières, telles que PWD, vous pouvez utiliser la commande "*printenv PWD*" ou "*env | grep PWD*".
- Utilisez la commande *export* pour définir et exporter des variables d'environnement.
- Utilisez la commande *unset* pour supprimer des variables d'environnement.

Exemple :

```
export MYVAR="Test"    # MYVAR est une var exportée
bash                   # Ouvre un nouveau shell (un sous-shell).
echo $MYVAR            # Affiche "Test"
```

2.2. Passage des variables d'environnement du processus parent au processus fils

Dans cette activité, nous étudions comment un processus fils obtient ses variables d'environnement de son parent. Sous Unix, la fonction **fork()** crée un nouveau processus en dupliquant le processus appelant. Le nouveau processus, appelé processus fils, est une copie exacte du processus appelant, appelé parent. Cependant, plusieurs choses ne sont pas héritées par le fils (veuillez consulter le manuel de fork() avec la commande : man fork).

Dans ce qui suit, nous allons découvrir si les variables d'environnement du parent sont héritées par le processus fils ou non.

Étape 1. Compilez et exécutez le programme suivant, et décrire votre observation. Le programme est dans le dossier Labsetup.

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
extern char **environ;
void printenv()
{
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
      printenv();          (1)
      exit(0);
    default:  /* parent process */
      // printenv();        (2)
      exit(0);
  }
}
```

Code 1 : myprintenv.c

Étape 2. Maintenant, commentez l'instruction printenv() dans le cas du processus fils (Ligne 1), et décommentez l'instruction printenv() dans le cas du processus parent (Ligne 2).

Compilez et exécutez à nouveau le code. Sauvegardez la sortie dans un autre fichier.

Étape 3. Comparez la sortie de ces deux fichiers en utilisant la commande diff.

Que remarquez-vous ?

2.3. Variables d'environnement et execve()

Dans cette activité, nous verrons comment les variables d'environnement sont affectées lorsqu'un nouveau programme est exécuté via la fonction execve().

Cette fonction fait un appel système pour charger une nouvelle commande et l'exécuter, elle ne fait aucun retour sauf en cas d'erreur. Aucun nouveau processus n'est créé ; au lieu de cela, le texte, les données et la pile du processus appelant sont écrasés par ceux du programme chargé. Particulièrement, execve() exécute le nouveau programme à l'intérieur du processus appelant. Dans ce qui suit, nous nous intéressons à ce qui arrive aux variables d'environnement ; sont-elles automatiquement héritées par le nouveau programme ?

Étape 1. Compilez et exécutez le programme ci-dessous, et décrire votre observation.

Ce programme exécute simplement un programme appelé /usr/bin/env, qui affiche les variables d'environnement du processus actuel.

```
#include <unistd.h>
extern char **environ;
int main()
{ char *argv[2];
  argv[0] = "/usr/bin/env";
  argv[1] = NULL;
  execve("/usr/bin/env", argv, NULL);      (1)
  return 0 ;
}
```

Code 2 : myenv.c

Étape 2. Changez l'appel de execve() dans la ligne (1) par ce qui suit et décrivez votre observation.

```
execve("/usr/bin/env", argv, environ) ;
```

Étape 3. Expliquez comment le nouveau programme récupère son environnement.

2.4. Variables d'environnement et system()

La fonction system() est utilisée pour exécuter une commande, mais contrairement à execve(), qui exécute directement une commande, system() exécute en fait "/bin/sh -c command", c'est-à-dire qu'elle exécute /bin/sh, et demande au shell d'exécuter la commande. Par conséquent, en utilisant system(), les variables d'environnement du processus appelant sont transmises au nouveau programme /bin/sh.

Veuillez compiler et exécuter le programme suivant pour vérifier ceci.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
        system("/usr/bin/env");
        return 0 ;
}
```

Quelle est votre observation ?

2.5. Variables d'environnement et programmes Set-UID

Set-UID est un mécanisme de sécurité essentiel dans les systèmes Unix. Lorsqu'un programme marqué Set-UID s'exécute, il adopte les privilèges de son propriétaire, et non ceux de l'utilisateur qui le lance. Ainsi, si le programme appartient à root, toute personne qui l'exécute lui fait temporairement acquérir les privilèges de root pendant son exécution. Ce mécanisme permet de réaliser des opérations privilégiées utiles, mais il est aussi potentiellement dangereux car il élève les droits de l'utilisateur.

Bien que le comportement d'un programme Set-UID est défini par sa logique interne et non directement par l'utilisateur, ce dernier peut tout de même influencer son exécution à travers les variables d'environnement. Pour analyser cet impact, il faut d'abord vérifier si le processus du programme Set-UID hérite bien les variables d'environnement du processus qui l'a lancé.

Étape 1. Écrivez ce programme qui affiche les variables d'environnement du processus en cours.

```
#include <stdio.h>
#include <stdlib.h>
extern char **environ;
int main()
{
        int i = 0;
        while (environ[i] != NULL) {
                printf("%s\n", environ[i]);
                i++;
        }
}
```

Étape 2. Compilez le programme ci-dessus, changez son propriétaire en root, et faites-en un programme Set-UID.

```
// Supposons que le nom du programme est « new ».
$ sudo chown root new
$ sudo chmod 4755 new
```

- Vérifiez, après chaque commande, les permissions du fichier new.
- Quelle est votre remarque après la deuxième commande ? Que signifie la valeur 4755 ?

Étape 3. Dans votre shell (en étant dans un compte utilisateur normal, pas dans le compte root), utilisez la commande *export* pour définir les variables d'environnement suivantes (elles peuvent déjà exister) :

- PATH="/bin:/usr/bin"
- LD_LIBRARY_PATH="Mylibrarypath"
- MY_VAR="..." (c'est une variable d'environnement définie par vous).

Ces variables d'environnement sont définies dans le processus du shell de l'utilisateur. Maintenant, exécutez le programme Set-UID de l'étape 2 dans votre shell. Après avoir tapé le nom du programme dans votre interpréteur de commandes, celui-ci s'exécute avec un **UID effectif différent**.

Veuillez vérifier si toutes les variables d'environnement que vous avez définies dans le processus shell (parent) se retrouvent dans le processus fils Set-UID.

- Qu'est-ce que vous remarquez ?

2.6. La variable d'environnement PATH et les programmes Set-UID

L'appel de system() dans un programme Set-UID peut être assez dangereux. Cela est dû au fait que le comportement réel du programme shell peut être affecté par des variables d'environnement, telles que PATH ; ces variables d'environnement sont fournies par l'utilisateur, qui peut être malveillant.

En modifiant ces variables, les utilisateurs malveillants peuvent contrôler le comportement du programme Set-UID.

Dans Bash, vous pouvez modifier la variable d'environnement PATH de la manière suivante (cet exemple ajoute le répertoire /home/seed au début de la variable d'environnement PATH) :

```
$ export PATH=/home/seed:$PATH
```

Le programme Set-UID ci-dessous est censé exécuter la commande /bin/ls; cependant, le programmeur utilise uniquement le chemin relatif pour la commande ls, plutôt que le chemin absolu:

```
int main()
        {
                system("ls");
                return 0;
        }
```

Étape 1. Compilez et exécutez ce programme.

- **a)** Quel est le résultat d'exécution de ce programme ?
- **b)** Quel est son propriétaire ?
- **c)** Donner les droits d'accès associés au : propriétaire, au groupe et aux autres utilisateurs.

Étape 2. Changez son propriétaire en root, et faites-en un programme Set-UID :

- **a)** Donner les commandes correspondantes.
- **b)** Vérifier les détails du programme.

Étape 3. Nous voulons faire en sorte que ce programme Set-UID exécute notre propre code malveillant, au lieu de /bin/ls :

- **a)** Comment peut-on le faire ?
- **b)** Si vous y parvenez, donnez le résultat d'exécution.
- **c)** Avec quel privilège s'exécute ce programme ? Justifiez.

Ci-dessous un prototype du programme malveillant :

```
// Code of the executable file named "ls" in the /home/seed directory:

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    printf("My malicious code!!\n");
    if (geteuid() == 0) {
        printf("I have root privilege!\n");
    }
    return 0;
}
```

Remarque importante :

La fonction *system (cmd)* exécute en premier lieu le programme */bin/sh*, puis demande à ce programme shell d'exécuter la commande *cmd*. Dans Ubuntu 20.04 (et plusieurs versions antérieures), /bin/sh est en fait un lien symbolique pointant vers /bin/dash. Ce programme shell dispose d'une contre-mesure qui l'empêche d'être exécuté dans un processus Set-UID. Donc, si *dash* détecte qu'il est exécuté dans un processus Set-UID, il remplace immédiatement l'ID utilisateur effectif par l'ID utilisateur réel du processus, supprimant essentiellement le privilège.

Puisque notre programme « victime » est un programme Set-UID, la contre-mesure dans /bin/dash peut empêcher notre attaque. Pour voir comment cette attaque fonctionne sans une telle contre-mesure, nous allons lier */bin/sh* à un autre shell qui ne dispose pas de cette contre-mesure. Nous allons installer un programme shell appelé *zsh* dans notre MV Ubuntu 20.04.

Utilisons la commande suivante pour lier */bin/sh* à */bin/zsh :*

```
$ sudo ln -sf /bin/zsh /bin/sh
```

Pour restaurer /bin/sh :

```
$ sudo ln -sf /bin/dash /bin/sh  # pour Debian/Ubuntu
```

2.7. Appel de programmes externes à l'aide de system() et execve()

Bien que system() et execve() permettent tous deux d'exécuter de nouveaux programmes, l'utilisation de system() dans un programme privilégié, comme un programme Set-UID, présente un risque important. En effet, nous avons vu comment la variable d'environnement PATH influence directement le comportement de system(), car elle impacte l'exécution du shell. À l'inverse, execve() ne souffre pas de cette vulnérabilité, puisqu'elle exécute directement le programme sans passer par le shell.

Mais, l'invocation du shell a une autre conséquence dangereuse, et cette fois, cela n'a rien à voir avec les variables d'environnement. Examinons le scénario suivant :

Supposant que Bob travaille pour une agence d'audit et il doit enquêter sur une entreprise pour une fraude présumée. Aux fins de l'enquête, Bob doit être capable de lire tous les fichiers du système de l'entreprise ; en revanche, pour protéger l'intégrité du système, Bob ne doit pouvoir modifier aucun fichier. Pour atteindre cet objectif, Alice, en tant que super-utilisateur du système, a écrit un programme spécial (voir ci-dessous), puis a donné l'autorisation exécutable à Bob. Ce programme demande à Bob de saisir un nom de fichier sur la ligne de commande, puis il exécutera /bin/cat pour afficher le fichier spécifié. Puisque le programme s'exécute en tant que root, il peut afficher n'importe quel fichier spécifié par Bob. Cependant, comme le programme n'a pas d'opérations d'écriture, il est certain que Bob ne pourra pas utiliser ce programme spécial pour modifier un fichier.

```
int main(int argc, char *argv[])
{

        char *v[3];
        char *command;
        if(argc < 2) {
          printf("Please type a file name.\n");
          return 1;
        }
        v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;
        command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
        sprintf(command, "%s %s", v[0], v[1]);
        // Use only one of the followings.
        system(command);
        // execve(v[0], v, NULL);
        return 0 ;

}
```

Code 3 : catall.c

Étape 1. Compilez le programme ci-dessus et testez-le sur un fichier texte. Le programme utilisera system() pour appeler la commande. Si vous étiez Bob, pouvez-vous compromettre l'intégrité du système ? Par exemple, pouvez-vous écrire sur un fichier qui n'est pas accessible en écriture ?

Étape 2. Commentez l'instruction system(command) et décommentez l'instruction execve() ; le programme utilisera execve() pour appeler la commande. Compilez le programme et exécutez-le sur le même fichier de l'étape 1. Votre attaque fonctionne-elle toujours ? Veuillez décrire et expliquer vos observations.

2.8. Fuite de capacité (Capability Leaking)

Pour respecter le principe du **moindre privilège**, les programmes **Set-UID** renoncent souvent à leurs privilèges **root** de manière définitive lorsque ces privilèges ne sont plus nécessaires. En outre, il arrive que le programme doive céder son contrôle à l'utilisateur ; dans ce cas, les privilèges root doivent être révoqués. L'appel système setuid() peut être utilisé pour révoquer les privilèges. Selon le manuel, "setuid() définit l'ID utilisateur effectif du processus appelant. Si l'UID effectif de l'appelant est root, l'UID réel et le set-user-ID sauvegardé sont également définis".

Par conséquent, si un programme Set-UID dont l'UID effectif est 0 appelle setuid(n), le processus deviendra un processus normal, tous ses UID étant définis sur n.

Lors de la révocation du privilège, l'une des erreurs les plus courantes est la fuite de capacités. Le processus peut avoir acquis certaines capacités lorsqu'il était encore privilégié ; lorsque le privilège est dégradé, si le programme ne nettoie pas ces capacités, elles peuvent encore être accessibles par le processus non privilégié. En d'autres termes, bien que l'ID utilisateur effectif du processus devienne non privilégié, le processus reste privilégié car il possède des capacités privilégiées.

Compilez le programme suivant, changez son propriétaire en root et faites-en un programme Set-UID. Exécutez le programme en tant qu'utilisateur normal. Pouvez-vous exploiter la vulnérabilité de fuite de capacité de ce programme ? Le but est d'écrire dans le fichier /etc/zzz en tant qu'utilisateur normal.

```
void main()
{

        int fd;
        char *v[2];
        /* Assume that /etc/zzz is an important system file,
        * and it is owned by root with permission 0644.
        * Before running this program, you should create
        * the file /etc/zzz first. */
        fd = open("/etc/zzz", O_RDWR | O_APPEND);
        if (fd == -1) {
          printf("Cannot open /etc/zzz\n");
          exit(0);
        }
        // Print out the file descriptor value
        printf("fd is %d\n", fd);
        // Permanently disable the privilege by making the
        // effective uid the same as the real uid
        setuid(getuid());
        // Execute /bin/sh
        v[0] = "/bin/sh"; v[1] = 0;
        execve(v[0], v, 0);

}
```

Code 4: cap_leak.c